Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет програмних систем та прикладної математики
Кафедра системного програмування і спеціалізованих комп’ютерних систем

ЛАБОРАТОРНА РОБОТА №2

з дисципліни

Розробка програмного забезпечення систем реального часу

Тема: Комунікаційні інтерфейси та якість обслуговування (QoS)

Виконала: Сапожко В.Ю.
Студентка IV курсу групи КВ-31
Перевірив(ла):

Київ 2026

GitHub: https://github.com/SValeriiiiaa/Real-Time-Software-Development-LW2-KV-31-Sapozhko

Мета роботи

Навчитися проектувати топіки, асинхронні сервіси та дії (Actions) в ROS 2, конфігурувати сумісні політики QoS (Quality of Service), реалізовувати «ледачу» публікацію даних (Lazy Publishers) та оптимізувати обмін даними за допомогою механізму адаптації типів (TypeAdaptation).

Хід виконання роботи

1. Створення та збирання робочого пакета

Для реалізації завдань лабораторної роботи у робочому просторі ROS 2 було створено пакет data_filter_pkg на базі системи збирання ament_cmake із залежностями від бібліотек rclcpp та std_msgs.

```
root@af98c0c6eb83:/ros2_ws# ls /ros2_ws/src
my_component_pkg
root@af98c0c6eb83:/ros2_ws# cd /ros2_ws/src
ros2 pkg create --build-type ament_cmake data_filter_pkg --dependencies rclcpp std_msgs
going to create a new package
package name: data_filter_pkg
destination directory: /ros2_ws/src
package format: 3
version: 0.0.0
description: TODO: Package description
maintainer: ['root <root@todo.todo>']
licenses: ['TODO: License declaration']
build type: ament_cmake
dependencies: ['rclcpp', 'std_msgs']
creating folder ./data_filter_pkg
creating ./data_filter_pkg/package.xml
creating source and include folder
creating folder ./data_filter_pkg/src
creating folder ./data_filter_pkg/include/data_filter_pkg
creating ./data_filter_pkg/CMakeLists.txt

[WARNING]: Unknown license 'TODO: License declaration'.  This has been set in the package.xml, but no LICENSE file has been created.
It is recommended to use one of the ament license identitifers:
Apache-2.0
BSL-1.0
BSD-2.0
BSD-2-Clause
BSD-3-Clause
GPL-3.0-only
LGPL-3.0-only
MIT
MIT-0
root@af98c0c6eb83:/ros2_ws/src#
```

Рисунок 1 - Створення пакета data_filter_pkg за допомогою утиліти ros2 pkg

Після реалізації вихідного коду вузла збирання пакета виконано за допомогою утиліти colcon із прапорцем --symlink-install для пришвидшення ітерацій розробки, після чого середовище було активовано через source-скрипт.

```
docker run -it --rm -v $(pwd)/ros2_ws:/ros2_ws ros2_lab1_image
root@52e6209c4b68:/ros2_ws# colcon build --packages-select data_filter_pkg --symlink-install
source install/setup.bash
Starting >>> data_filter_pkg
Finished <<< data_filter_pkg [0.30s]

Summary: 1 package finished [0.48s]
```

Рисунок 2 - Успішне збирання пакета data_filter_pkg утилітою colcon

2. Реалізація механізму «ледачої» публікації (Lazy Publisher) та обробки даних

Було розроблено вузол data_filter, який підписується на вхідний топік /input_topic типу std_msgs/msg/String, виконує перетворення тексту у верхній регістр та транслює його у топік /uppercase_topic.

Для оптимізації використання обчислювальних ресурсів у вузлі реалізовано механізм Lazy Publisher на основі зворотного виклику matched_callback:

- При запуску вузла підписка на вхідний потік /input_topic не створюється (Unknown topic), доки на виході немає активних споживачів.
- При підключенні підписника на топік /uppercase_topic спрацьовує matched_callback, який динамічно ініціалізує підписку на вхідні дані.
- При відключенні останнього слухача (кількість дорівнює 0) підписка на вхідний топік анулюється, запобігаючи зайвій обробці даних. Вихідний видавець налаштовано із політикою тривалості transient_local та глибиною черги 1, що забезпечує збереження та передачу останнього актуального повідомлення новим підписникам.

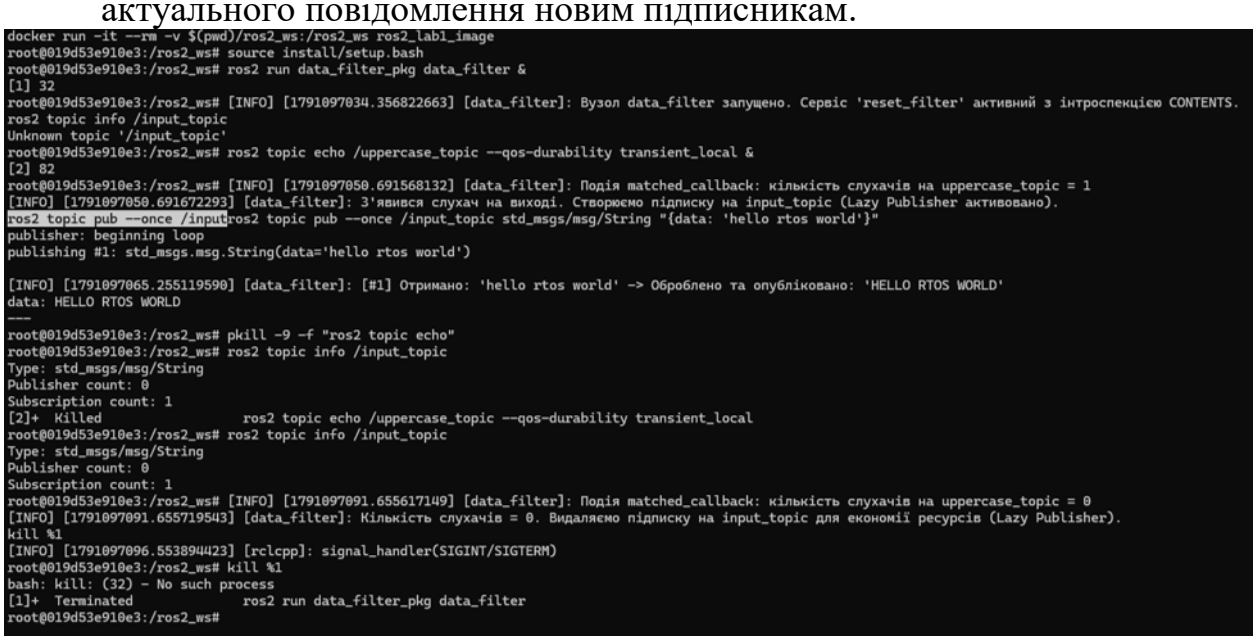


Рисунок 3 - Демонстрація механізму Lazy Publisher та обробки рядкових повідомлень вузлом data_filter

3. Перевизначення політик якості обслуговування (QoS Overriding)

Для забезпечення гнучкості налаштування комунікаційних каналів без перекомпіляції коду до видавця топіка /uppercase_topic було додано підтримку rclcpp::QosOverridingOptions.

Перевірка списку параметрів вузла за допомогою команди ros2 param list підтвердила наявність параметрів перевизначення глибини (depth), надійності (reliability) та збереження (durability). За замовчуванням видавець транслював дані з політикою RELIABLE.

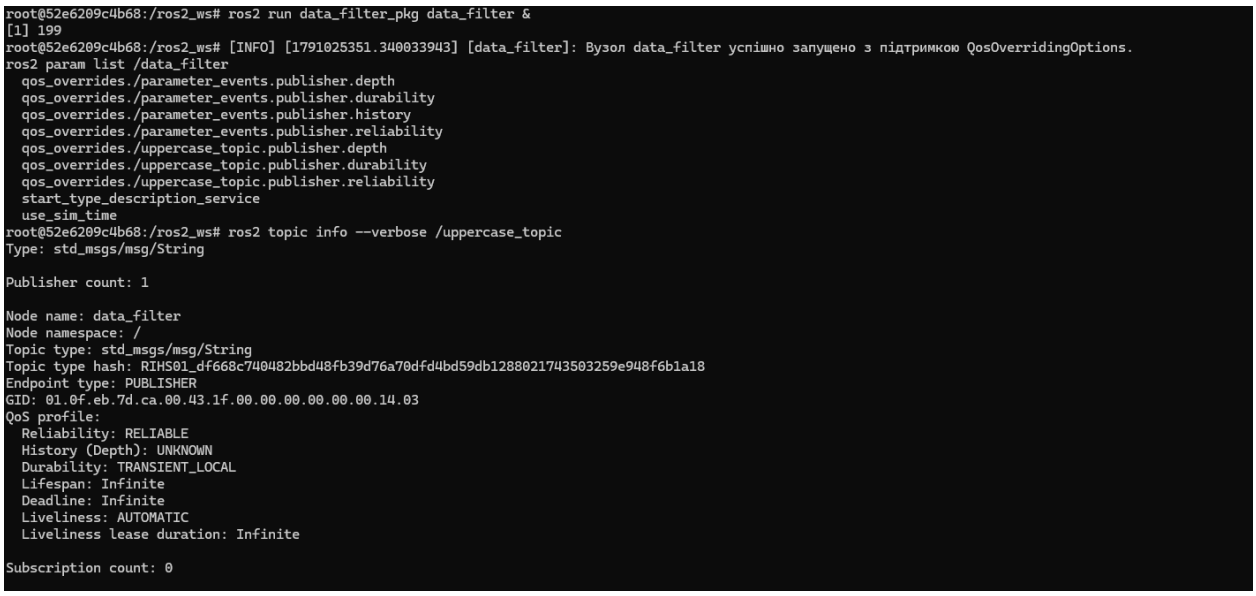


Рисунок 4 - Перевірка доступних параметрів QoS-перевизначення та початкового профілю топіка uppercase_topic

Для перевірки функціоналу динамічного керування політиками вузол було перезапущено із передачею нових значень через аргументи командного рядка: надійність змінено на best_effort, а розмір черги повідомлень — на 5. Детальний перегляд параметрів топіка через ros2 topic info --verbose підтвердив успішне застосування нових налаштувань QoS у системі.

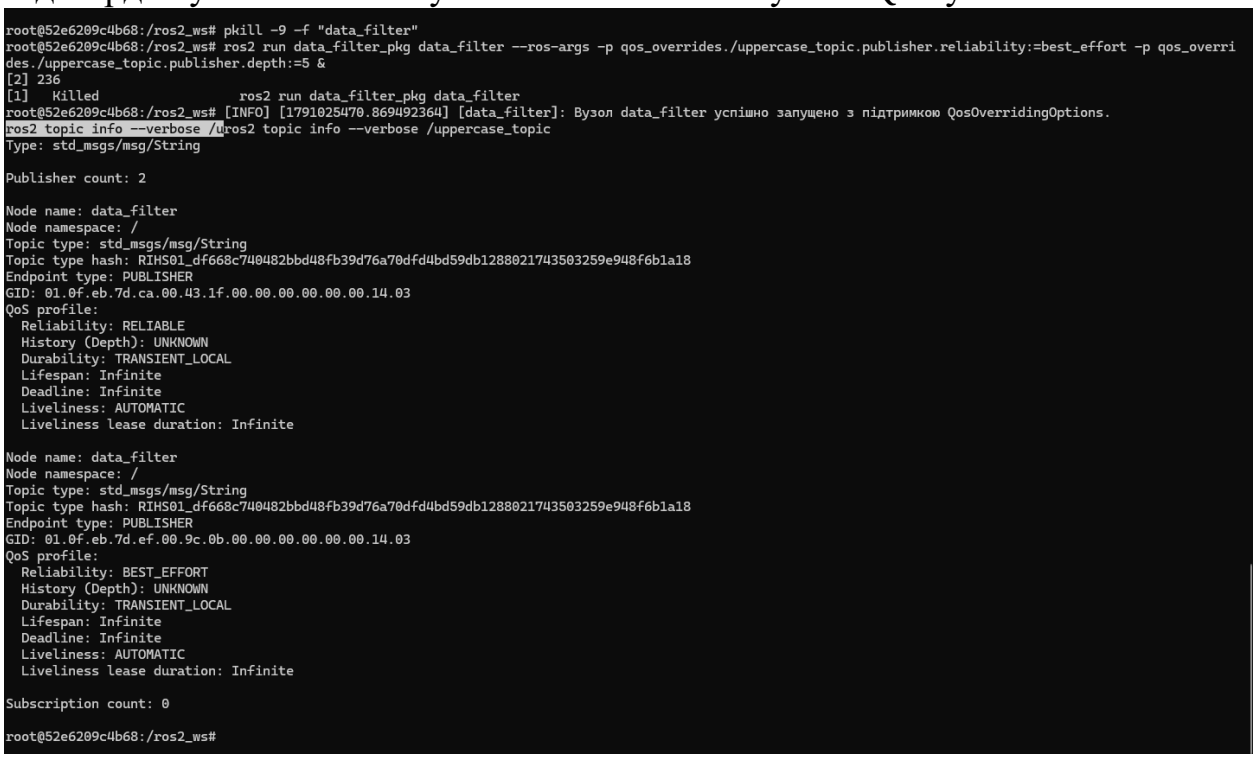


Рисунок 5 - Перевизначення політик QoS через параметри запуску та верифікація зміненого профілю видавця

4. Реалізація асинхронного сервісу та інтроспекція подій

У вузол було інтегровано сервісний сервер ResetFilter, призначений для скидання лічильника оброблених повідомлень. Для клієнта було реалізовано асинхронний виклик async_send_request із безпечним очікуванням відповіді за таймаутом через метод wait_for.

Для моніторингу роботи сервісу було активовано вбудований механізм інтроспекції за допомогою методу configure_introspection зі станом RCL_SERVICE_INTROSPECTION_CONTENTS. Наявність службового топіка підтверджено викликом команди ros2 topic list --include-hidden-topics, де відобразився топік /reset_filter/_service_event.

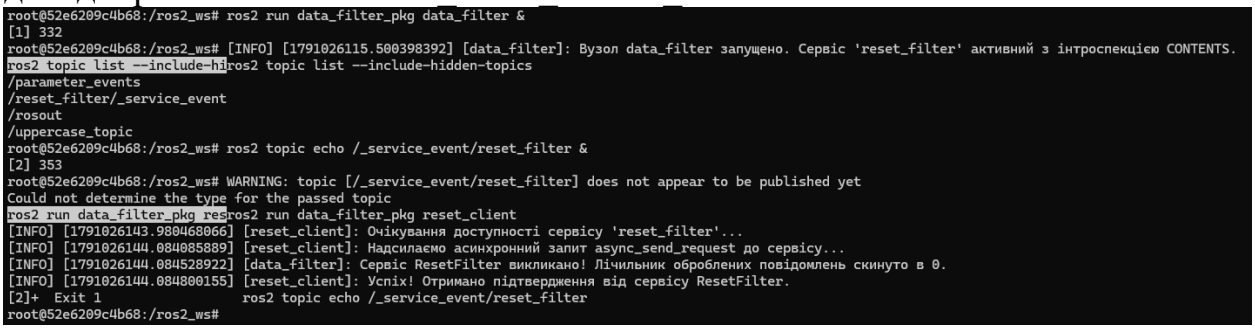


Рисунок 6 - Відображення прихованого топіка інтроспекції та успішне виконання асинхронного запиту клієнтом

Через підписку на топік подій сервісу за допомогою утиліти ros2 topic echo було перехоплено метадані та тіло повідомлень. При відправці команди клієнтом інтроспекція зафіксувала подію отримання відповіді (event_type: 2) разом із міткою часу та ідентифікатором клієнта.

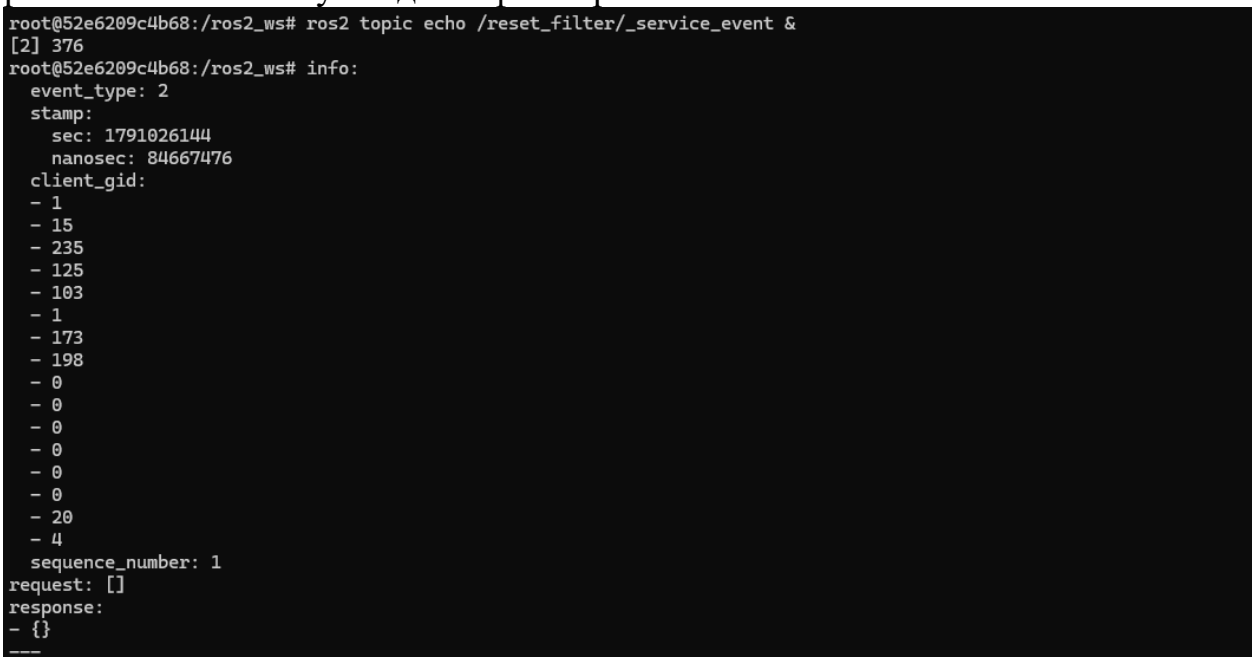


Рисунок 7 - Перехоплення події відповіді сервісу в топіку reset_filter/_service_event

Повний захват подій продемонстрував реєстрацію обох фаз комунікації: передачу клієнтського запиту (event_type: 1 - REQUEST) та повернення

відповіді сервером (event_type: 2 - RESPONSE) з повним розкриттям структури даних.



Рисунок 8 - Фіксація подій запиту (Request) та відповіді (Response) сервісу в потоці інтроспекції



Рисунок 9 - Завершення перехоплення структури відповіді сервісу в топіку інтроспекції

5. Створення інтерфейсу та сервера дій (Action Server)

Для виконання тривалих обчислювальних операцій було створено окремий пакет інтерфейсів data_filter_interfaces, у якому описано дію ProcessData.action. Структура інтерфейсу містить:

Ціль (Goal): вхідний масив чисел int32[] data;

Результат (Result): результуючий масив int32[] result_data;

Зворотний зв'язок (Feedback): відсоток виконання завдання int32 progress.

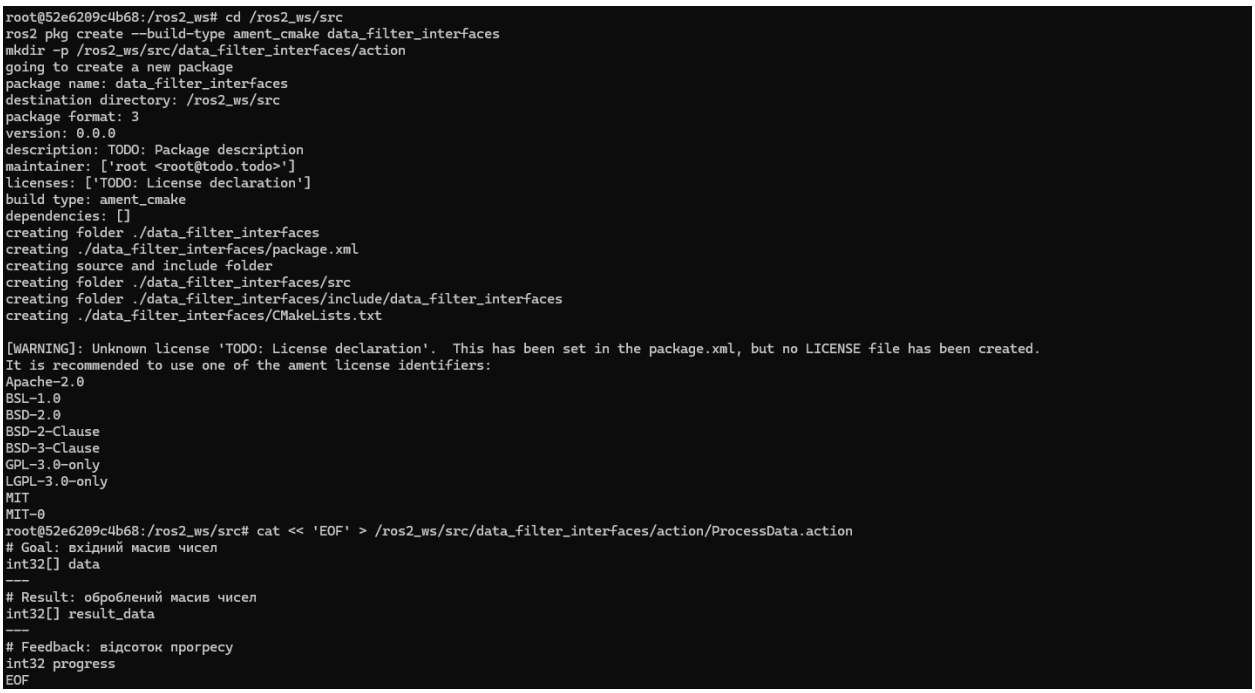


Рисунок 10 - Оголошення структури інтерфейсу дії у файлі ProcessData.action

Було розроблено вузол action_server, який приймає запит на обробку масиву, переносить важкі обчислення у фоновий асинхронний потік, періодично інформує клієнта про стан прогресу та формує вихідний масив.

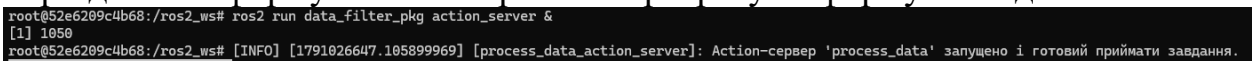


Рисунок 11 - Запуск Action-сервера process_data_action_server

Тестування дії виконано через CLI за допомогою команди ros2 action send_goal із прапорцем --feedback. Сервер успішно прийняв ціль, виконав поелементну обробку з публікацією проміжного прогресу (від 20% до 100%) та повернув подвоєні значення елементів масиву із фінальним статусом SUCCEEDED.

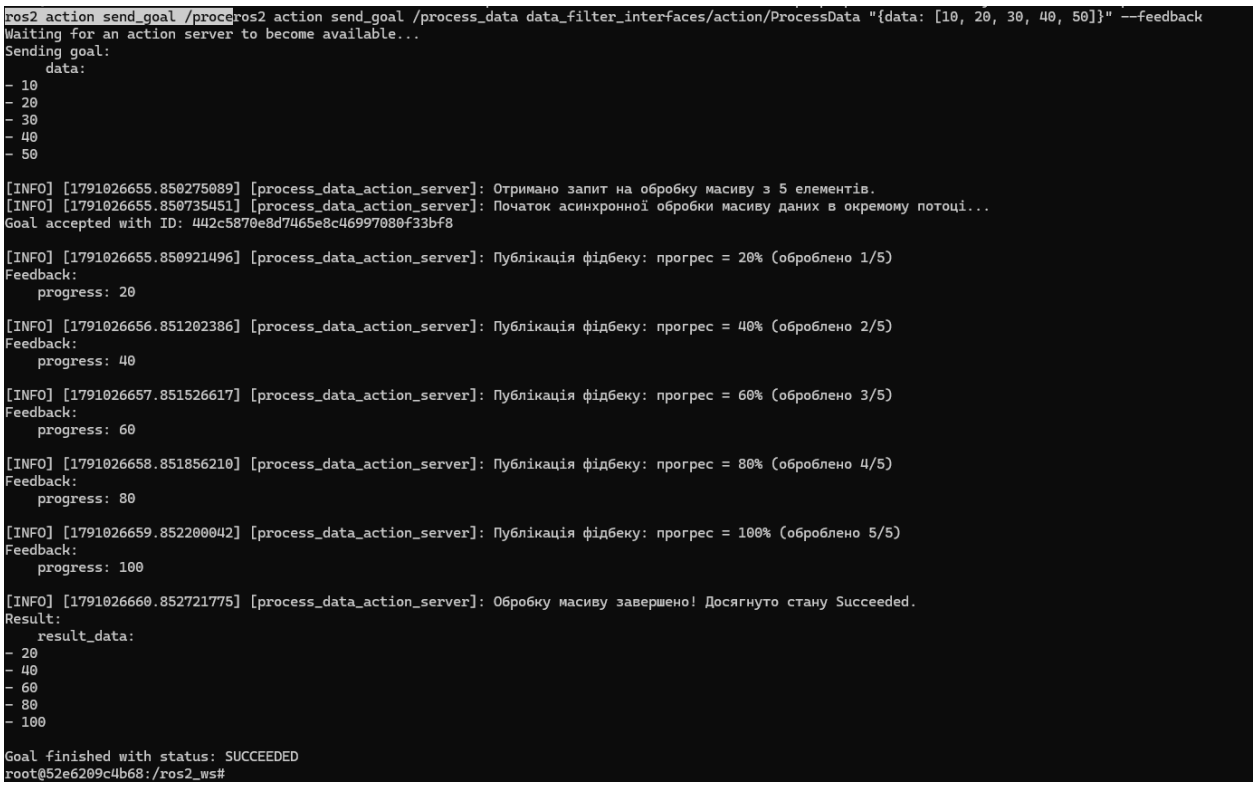


Рисунок 12 - Відправка цілі, отримання періодичного зворотного зв'язку (feedback) та успішне завершення дії

6. Оптимізація обміну повідомленнями за допомогою Type Adaptation (REP-2007)

Для усунення накладних витрат під час передачі великих масивів даних (на прикладі сенсорних хмар точок sensor_msgs/msg/PointCloud2) було реалізовано адаптер типів за допомогою шаблону rclcpp::TypeAdapter.

Було проведено порівняльний аналіз передачі даних у двох режимах:

1. **Звичайна міжпроцесна взаємодія (--no-intra):** при передачі даних щоразу викликаються методи адаптера convert_to_ros_message та convert_to_custom для маршалінгу/демаршалінгу структури. Адреси пам'яті об'єкта у видавця (0x62ba54bee440) та отримувача (0x62ba54bef4c0) відрізняються, що свідчить про фізичне копіювання буфера пам'яті.
2. **Внутрішньопроцесна комунікація (--intra):** при увімкненні intra-process communication та передачі через розумні вказівники std::unique_ptr адаптаційні методи конвертації взагалі не викликаються, а адреси пам'яті вказівників у продюсера та консюмера є ідентичними (0x5d3b86a01ab0). Це підтверджує реалізацію концепції нульового копіювання (Zero-Copy), яка є критичною для систем реального часу.

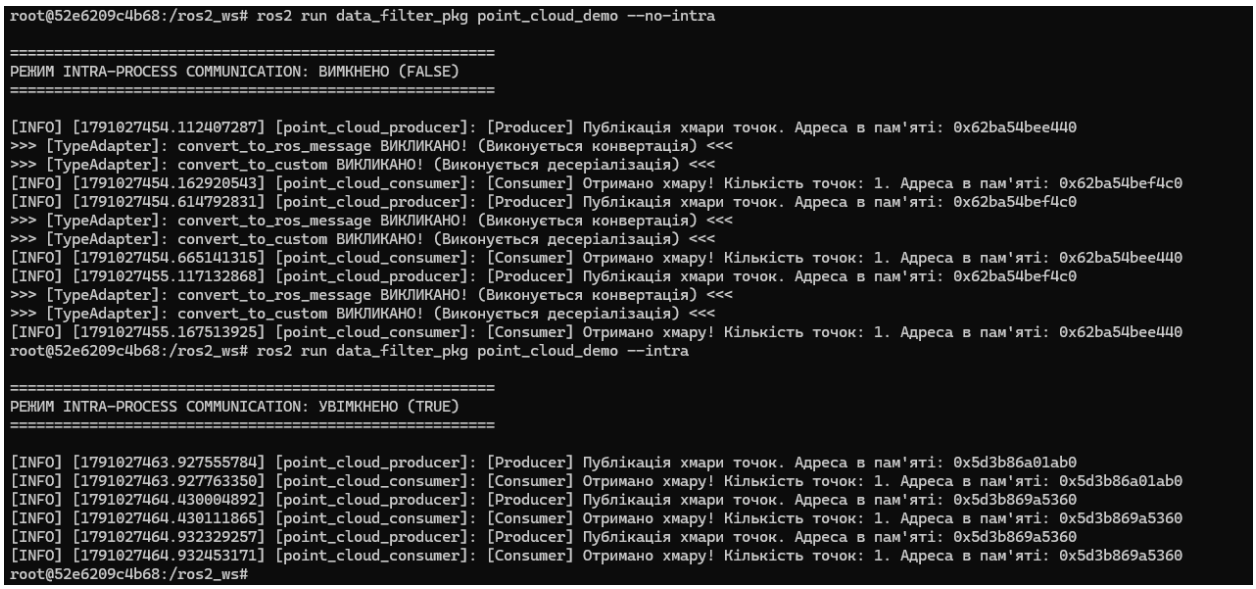


Рисунок 13 - Порівняння передачі даних без оптимізації та в режимі нульового копіювання (Type Adaptation Zero-Copy)

Висновок

Під час виконання лабораторної роботи було опановано проектування та конфігурування ключових комунікаційних інтерфейсів у середовищі ROS 2:

1. Досліджено механізм «ледачої» публікації (Lazy Publishing) за допомогою matched_callback, що дозволяє оптимізувати навантаження на процесор у системах реального часу шляхом динамічного керування підписками залежно від наявності споживачів.
2. Застосовано політики якості обслуговування (QoS), зокрема профіль transient_local для збереження останнього стану даних, а також налаштовано динамічне перевизначення параметрів QoS у режимі виконання за допомогою rclcpp::QosOverridingOptions.
3. Розроблено асинхронний сервіс із контролем таймауту та на практиці досліджено новий механізм інтроспекції сервісів через приховані службові теми /_service_event, що дозволяє здійснювати повний аудит викликів і відповідей у розподіленій системі.
4. Створено Action-сервер для тривалих асинхронних завдань, що забезпечує неблокуючу фонову обробку потоків даних та інформування клієнта через канал зворотного зв'язку (feedback).
5. Реалізовано адаптацію типів даних (rclcpp::TypeAdapter згідно з REP-2007) та експериментально підтверджено ефект нульового копіювання пам'яті (Zero-Copy) при внутрішньопроцесному обміні, що мінімізує латентність передачі великих структур (хмар точок).